mini-scope – Troubleshooting Guide

Finding the cause when mini-scope does not work on your board

October 22, 2026

1. Before the first start

Most problems come from one of these points. Check them first:

Check	Why
Program a **production** build (“Make and Program Device”), not a debug build	With `ICS = PGD3`, the debug executive uses RB5/RB6, the same pins as mini-scope.
Disconnect the PICkit after programming	Its pull-downs on PGC/PGD and its drivers disturb the UART lines.
`scope_init()` is the **last** call before the main loop	Pin and PPS setup done later in your code can overwrite the mini-scope pins. `scope_init()` also enables interrupts globally.
`SCOPE_FP_HZ` equals FOSC/2 of your clock setup	The baud rate is derived from it. With FOSC = 64 MHz, FP is 32 MHz.
UART and timer selected in `scope.h` are **not used** by your application	`scope_init()` reconfigures them.
ICSP header pin 1 (MCLR) and pin 2 (3V3) stay **unconnected**	A data line on MCLR resets the controller with every character.
The board has its own supply	The USB-UART adapter does not power the board.

mini-scope has been checked against the data sheet, in the MPLAB X simulator and in a PC model, but **not yet on real hardware**. If you find a problem that is not covered here, please report it (see section 9).

2. Build errors

Message	Cause	Remedy
`multiple definition of _T1Interrupt` (or `_U1RXInterrupt`, `_U1TXInterrupt`, `_CCT1Interrupt` …)	Your application already uses this timer or UART.	Select a free one in `scope.h`: `SCOPE_UART` 1, 2 or 3; `SCOPE_TIMER` `SCOPE_T1`, `SCOPE_CCP1` … `SCOPE_CCP4` or `SCOPE_NONE`.
`#error "SCOPE_UART must be 1, 2 or 3"` / `"SCOPE_TIMER must be …"`	Invalid selection	Use one of the listed values.
`#error "SCOPE_CAPBUF must be 64..65535 bytes"` / `"SCOPE_TXBUF …"` / `"SCOPE_MAX_CH …"`	Buffer setting out of range	See the RAM table in `README.md`.
Linker: data memory full, or `region 'data' overflowed`	mini-scope (about 4.7 KB by default) plus your variables exceed the 8 KB RAM.	Reduce `SCOPE_CAPBUF`, `SCOPE_TXBUF`, `SCOPE_MAX_CH`. Example: 1024 / 256 / 8 needs about 1.4 KB.
`'_U1RXR' undeclared` or similar	Wrong device selected in the project	mini-scope is written for the dsPIC33CK64MC10x family.

3. Starting the user interface

Symptom	Remedy
"Python 3.9 or newer was not found"	Install Python from python.org and tick "Add python.exe to PATH". Then run `start_gui.bat` again.
"Package installation failed"	The first start downloads packages. Behind a proxy, run `set HTTPS_PROXY=http://proxy:port` in a console, then `start_gui.bat` from the same console. If the company blocks pip, ask IT to allow pypi.org.
Browser does not open	Open http://localhost:8080 by hand.
Error "address already in use"	Another program uses port 8080. Start with `start_gui.bat --port 8081` and open http://localhost:8081.
Windows Firewall asks whether to allow Python	The user interface only needs local access. Choose **Cancel**/**Deny**; the browser on the same PC still works.
Browse ... shows no dialog	The file dialog may open behind the browser window; check the taskbar. If Python was installed without Tcl/Tk, type the path into **ELF file** and click **Load**.
COM port busy when programming or using another terminal	Click **Disconnect** first; it releases the port.
The environment seems broken after a Python update	Delete the folder `.venv` in the project folder. The next start recreates it.

4. "port open, but no answer from target"

The PC can open the COM port, but the controller does not answer. Work through these steps in order.

4.1 Port and wiring

1. Is the right COM port selected? Unplug the adapter, reload the page (F5) and see which port disappears.
2. Is the PICkit disconnected?
3. Are RX and TX crossed? The controller's TX (ICSP pin 4, PGD) goes to the adapter's **RX**, the controller's RX (pin 5, PGC) to the adapter's **TX**, pin 3 to GND. If in doubt, swap the two lines; nothing breaks.
4. Is the adapter a 3.3 V type? A 5 V or RS-232 adapter does not work.

4.2 Is the firmware running at all?

- Was a **production** build programmed? A debug build waits for the debugger, or occupies the pins.
- Is `scope_init()` actually reached? A wait loop on a clock flag (PLL lock) that never ends, or a watchdog reset loop, stops it before. A toggling LED or test pin in the main loop shows whether the main loop runs.

4.3 Is the clock what `SCOPE_FP_HZ` says?

If FP differs from `SCOPE_FP_HZ`, the controller sends at the wrong baud rate. You can find out without an oscilloscope: in the user interface, set **Baud** to the values below one after the other and click **Connect** each time. The rate that connects tells you the real FP:

Baud that connects	Real FP	Set in `scope.h`
115200	32 MHz	nothing to change

Baud that connects	Real FP	Set in `scope.h`
57600	16 MHz	`SCOPE_FP_HZ 16000000UL`
230400	64 MHz, faster than intended: check the PLL setup	`SCOPE_FP_HZ 64000000UL`
14400	4 MHz: the PLL is not running, the device runs on FRC (8 MHz / 2)	fix the PLL setup, or `SCOPE_FP_HZ 4000000UL`

In general, real FP = 32 MHz × (baud that connects / 115200). After changing `SCOPE_FP_HZ`, rebuild, program, and set **Baud** back to 115200.

With an oscilloscope: probe ICSP pin 4 while clicking **Connect**. At 115200 baud one bit lasts 8.7 µs. A bit time of 17.4 µs means FP is half of `SCOPE_FP_HZ`.

Note on the PLL: the dividers (`PLLFBDIV`, `POST1DIV`, `POST2DIV`) must not be changed while the PLL is the active clock. Start on FRC, set the dividers, then switch to FRCPLL and wait for `OSCCONbits.LOCK`. `example_main.c` shows the sequence.

4.4 Something else uses the pins or the UART

- Does your code write the whole `TRISB`, `ANSELB` or a PPS register (`RPORx`, `RPINR18/19/27`) **after** `scope_init()`? Then the UART pins are reconfigured. Move `scope_init()` to the end, or exclude bits 5 and 6.
- Does your code use the same UART without interrupts (polled)? There is no link error in this case, but both parts disturb each other. Select another UART with `SCOPE_UART`.
- With other pins than RB5/RB6: do `SCOPE_TX_RP` / `SCOPE_RX_RP` match the wiring? RB*n* is RP(32 + *n*), RC*n* is RP(48 + *n*).
- Are there other loads on the UART pins on your board (LEDs, pull resistors, filter capacitors)? Strong loads or large capacitors distort the signal.

4.5 Interrupts

mini-scope works entirely in interrupts (UART priority 1, sampling priority 2).

- Does your code disable interrupts for long periods, or run the CPU at an interrupt priority level above 2 (`SRbits.IPL`)? Then the mini-scope interrupts are blocked.
- Does your code clear `INTCON2bits.GIE` after `scope_init()`?

5. Connected, but the values are wrong

Symptom	Cause	Remedy
Curve is a flat line	wrong address, or the variable is not updated	**Load** the ELF again after every rebuild; addresses move. Check that the variable really changes, for example in the debugger.
Values make no sense	wrong type	The ELF only provides the size, not the type. Select `u16` for `uint16_t`, `f32` for `float` and so on.
Single values are wrong now and then	32-bit value read while it is being written, or a variable updated in several steps	A 32-bit value is read in two 16-bit steps. Write the value in one assignment from a local variable.

Symptom	Cause	Remedy
Value lags behind	variable kept in a CPU register by the optimizer	Declare it `volatile`.
Variable is not in the list	local variable, struct member, array, or larger than 4 bytes	Only global and static variables of 1, 2 or 4 bytes are listed. For a struct member, enter the address by hand: struct address + offset.
"Target rejected the configuration"	too many channels, or an odd address on a 16- or 32-bit channel	The maximum number of channels is `SCOPE_MAX_CH`. 16- and 32-bit values must be at even addresses.
"lost frames" counts up, orange "⚠ Too fast for the UART" warning	stream rate too high for 115200 baud (about 11 kB/s)	Use at least the sample period named in the warning, fewer channels, or Capture.
Time axis is wrong	`SCOPE_TIMER = SCOPE_NONE`, and the sample period in the GUI does not match the rate at which `scope_sample()` is called	Enter the period of the ISR that calls `scope_sample()`.

6. Capture does not work

Symptom	Cause	Remedy
Status stays at "captures: 0"	the trigger condition never occurs	The threshold is a **raw value**, without the factor. Check the trigger channel number (the first channel is 0) and the edge. Try edge **none**, which records immediately.
Trigger point not where expected	edge direction, or the signal crosses the threshold several times	The trigger fires at the first crossing after the pretrigger part is filled.
A recording takes about half a second to appear	normal: a full 4 KB buffer needs about 0.4 s at 115200 baud	Use a smaller `SCOPE_CAPBUF` for faster updates.
Recording is too short	buffer divided by the bytes of all channels	Fewer or smaller channels, or a larger `SCOPE_CAPBUF` if RAM allows.

7. The application behaves differently with mini-scope

Symptom	Cause	Remedy
Controller resets during a measurement	the watchdog: with very short sample periods the sampling interrupt leaves too little time for the main loop	Use a sample period of 20 µs or more, or `SCOPE_TIMER = SCOPE_NONE` and call `scope_sample()` from your control ISR.
Your UART or timer stops working	conflict with the module selected in `scope.h`	Select another UART or timer.
Interrupts fire earlier than before	`scope_init()` sets `GIE = 1`	Call `scope_init()` after all other initialisation.
Motor control jitters	mini-scope interrupts at priority 1/2 delay lower-priority code	Keep your time-critical ISRs above priority 2, or sample from your own ISR with `SCOPE_NONE`.

Symptom	Cause	Remedy
Noise, the connection drops when the motor runs	ground loop or interference through the USB-UART adapter	Use a galvanically isolated adapter, or run the laptop on battery, without docking station, mains adapter or LAN.

8. Simulator

Problems with `start_sim.bat` and the MPLAB X simulator are covered in `SIMULATOR.md`, section 9.

9. Asking for help

Please send the following:

1. The text of the status line and of any error message, or a screenshot.
2. The settings at the top of `scope.h` that you changed.
3. Your clock setup (configuration bits `FNOSC`, `FWDTEN`, `ICS` and the PLL code).
4. Which UART, timers and PPS assignments your application uses.
5. For build errors: the build output. For `start_sim.bat`: `test.X\build_sim.log`.
6. For wrong values: the CSV export and the type of the variable in your code.